

DeepMNavigate: Deep Reinforced Multi-Robot Navigation Unifying Local & Global Collision Avoidance

Qingyang Tan¹, Tingxiang Fan², Jia Pan², Dinesh Manocha¹

Video Link: <https://youtu.be/LWLBxWuwPeU>

Abstract—We present a novel algorithm (DeepMNavigate) for global multi-agent navigation in dense scenarios using deep reinforcement learning (DRL). Our approach uses local and global information for each robot from motion information maps. We use a three-layer CNN that takes these maps as input to generate a suitable action to drive each robot to its goal position. Our approach is general, learns an optimal policy using a multi-scenario, multi-state training algorithm, and can directly handle raw sensor measurements for local observations. We demonstrate the performance on dense, complex benchmarks with narrow passages and environments with tens of agents. We highlight the algorithm’s benefits over prior learning methods and geometric decentralized algorithms in complex scenarios.

I. INTRODUCTION

Multi-robot systems are increasingly being used for different applications, including surveillance, quality control systems, autonomous guided vehicles, warehouses, cleaning machines, etc. A key challenge is to develop efficient algorithms for navigating such robots in complex scenarios while avoiding collisions with each other and the obstacles in the environment. As larger numbers of robots are used, more efficient methods are needed that can handle dense and complex scenarios.

Multi-agent navigation has been studied extensively in robotics, AI, and computer animation. At a broad level, previous approaches can be classified into centralized [1], [2], [3], [4], [5], [6] or decentralized planners [7], [8], [9], [10]. One benefit of decentralized methods is that they can scale to a large number of agents, though it is difficult to provide any global guarantees on the resulting trajectories [11] or to handle challenging scenarios with narrow passages (see Figures 1 or 2).

Recently, there has been considerable work on developing new, learning-based planning algorithms [12], [13], [14], [15], [16], [17] for navigating one or more robots through dense scenarios. Most of these learning methods tend to learn an optimal policy using a multi-scenario, multi-stage training algorithm. However, current learning-based methods are limited to using only the local information and do not exploit any global information. Therefore, it is difficult to use them in dense environments or narrow passages.

Main Results: We present a novel, multi-robot navigation algorithm (DeepMNavigate) based on reinforcement learning that exploits a combination of local and global information. We use a multi-stage training scheme that uses various

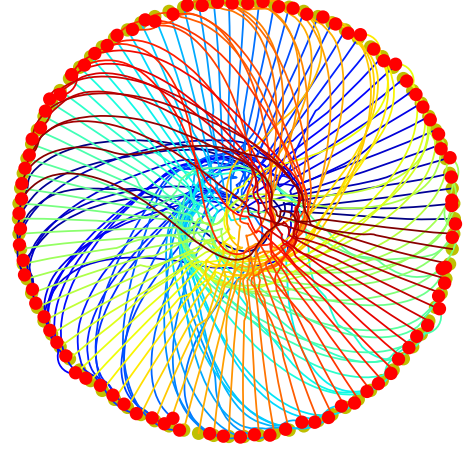


Fig. 1: **Circle Crossing:** Simulated trajectories of 90 robots in circle crossing scenarios generated by our algorithm that uses global information. The yellow points are the initial positions of the robots and the red points are the diametrically opposite goals of the robot. Our DeepMNavigate algorithm can handle such scenarios without collisions along the trajectories, and all the robots reach their goals. Prior learning methods that only use local methods [16], [12], [14] cannot handle such scenarios, as the robots tend to get stuck.

multi-robot simulation scenarios with global information. In terms of training, we represent the global information using a *motion information* map that includes the location of each agent or robot. We place the robot information in the corresponding position to generate a bit-map and use the resulting map as an input to a three-layer CNN. Our CNN considers this global information along with local observations in the scene to generate a suitable action to drive each robot to the goal without collisions. We have evaluated our algorithm in dense environments with many tens of robots (e.g., 90 robots) navigating in tight scenarios with narrow passages. As compared to prior multi-robot methods, our approach offers the following benefits:

- 1) We use global knowledge in terms of motion information maps in our network to improve the performance of DRL. This also results in higher reward value.
- 2) Our approach scales with the number of robots and is able to compute collision-free and smooth trajectories. Running our trained system takes many tens of seconds on a PC with a 32-core CPU and one NVIDIA RTX 2080 Ti on multi-robot systems with 10 – 90 robots.
- 3) We can easily handle challenging multi-robot scenarios like inter-changing robot positions or multiple narrow corridors, which are difficult for prior geometric decentralized or local learning methods. In particular, we highlight the performance on five difficult environments that are very different from our training

¹Qingyang Tan and Dinesh Manocha are with Department of Computer Science and Electrical & Computer Engineering, University of Maryland at College Park. {qytan,dm@cs.umd.edu} ²Tingxiang Fan and Jia Pan are with Department of Computer Science, University of Hong Kong. {tingxfan@hku.hk, jpan@cs.hku.hk}

Project website: https://qytan.com/publication/global_planning/

DeepMNavigate: 深度强化的多机器人导航统一局部和全局防撞

谭庆阳¹、范婷翔²、潘佳²、Dinesh Manocha¹ 视频链接: <https://youtu.be/LWLBxWuwPeU>

Abstract— 我们提出了一种新颖的算法 (DeepMNavigate), 用于使用深度强化学习 (DRL) 在密集场景中进行全局多智能体导航。我们的方法使用运动信息图中每个机器人的本地和全局信息。我们使用三层 CNN, 将这些地图作为输入, 生成合适的动作来驱动每个机器人到达其目标位置。我们的方法是通用的, 使用多场景、多状态训练算法学习最优策略, 并且可以直接处理本地观测的原始传感器测量结果。我们在狭窄通道和具有数十个智能体的环境中展示了密集、复杂的基准测试的性能。我们强调了该算法在复杂场景中相对于现有学习方法和几何分散算法的优势。

一、简介

多机器人系统越来越多地用于不同的应用, 包括监控、质量控制系统、自动导引车、仓库、清洁机等。一个关键的挑战是开发有效的算法来在复杂的场景中导航此类机器人, 同时避免相互碰撞以及环境中的障碍物。随着机器人数量的增加, 需要更有效的方法来处理密集和复杂的场景。

多智能体导航在机器人、人工智能和计算机动画领域得到了广泛的研究。在广泛的层面上, 以前的方法可以分为集中式[1]、[2]、[3]、[4]、[5]、[6]或分散式规划器[7]、[8]、[9]、[10]。去中心化方法的好处之一是它们可以扩展到大量代理, 尽管很难对结果轨迹提供任何全局保证 [11] 或处理具有狭窄通道的挑战性场景 (见图 1 或图 2)。

最近, 在开发基于学习的新规划算法 [12]、[13]、[14]、[15]、[16]、[17] 方面开展了大量工作, 用于在密集场景中导航一个或多个机器人。这些学习方法大多数倾向于使用多场景、多阶段的训练算法来学习最优策略。然而, 当前基于学习的方法仅限于仅使用局部信息, 并且不利用任何全局信息。因此, 很难在密集的环境或狭窄的通道中使用它们。

主要结果: 我们提出了一种基于利用 *a combination of local and global information* 的强化学习的新型多机器人导航算法 (DeepMNavigate)。我们采用多阶段训练方案, 使用各种

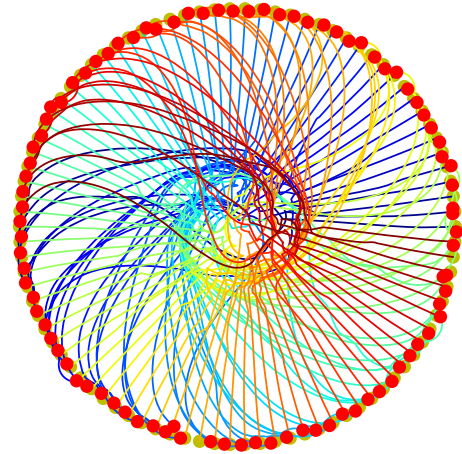


图 1: 圆圈交叉: 我们使用全局信息的算法生成的圆圈交叉场景中 90 个机器人的模拟轨迹。黄点是机器人的初始位置, 红点是机器人截然相反的目标。我们的 DeepMNavigate 算法可以处理此类场景, 而不会沿着轨迹发生碰撞, 并且所有机器人都达到了目标。仅使用本地方法的先前学习方法[16]、[12]、[14] 无法处理此类场景, 因为机器人往往会被卡住。

具有全局信息的多机器人模拟场景。在训练方面, 我们使用包含每个代理或机器人位置的 *motion information* 地图来表示全局信息。我们将机器人信息放置在相应的位置以生成位图, 并将生成的图用作三层 CNN 的输入。我们的 CNN 考虑全局信息以及场景中的局部观察, 以生成合适的动作来驱动每个机器人到达目标而不会发生碰撞。我们在密集环境中评估了我们的算法, 其中有数十个机器人 (例如 90 个机器人) 在狭窄通道的狭窄场景中导航。与之前的多机器人方法相比, 我们的方法具有以下优点:

- 1) 我们在网络中使用运动信息图方面的全局知识来提高 DRL 的性能。这也会带来更高的奖励价值。
- 2) 我们的方法随着机器人的数量而扩展, 并且能够计算无碰撞且平滑的轨迹。在具有 32 核 CPU 和 1 个 NVIDIA RTX 2080 Ti 的 PC 上, 在具有 10 个 – 90 个机器人的多机器人系统上运行我们经过训练的系统需要数十秒。
- 3) 我们可以轻松处理具有挑战性的多机器人场景, 例如互换机器人位置或多个狭窄走廊, 这对于先前的几何分散或局部学习方法来说是困难的。我们特别强调了在五种困难环境中的表现, 这些环境与我们训练有很大不同

¹Qingyang Tan and Dinesh Manocha are with Department of Computer Science and Electrical & Computer Engineering, University of Maryland at College Park. {qytan,dm@cs.umd.edu} ²Tingxiang Fan and Jia Pan are with Department of Computer Science, University of Hong Kong. {tingxfan@hku.hk, jpan@cs.hku.hk}

Project website: https://qytan.com/publication/global_planning/

scenarios and have more agents. This demonstrates the generalizability of our method.

II. RELATED WORK

A. Geometric Multi-Robot Navigation Algorithms

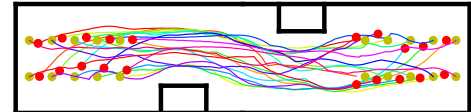
Most prior algorithms are based on geometric techniques such as sampling-based methods, geometric optimization, or complete motion planning algorithms. The centralized methods assume that each robot has access to complete information about the state of the other robots based on some global data structure or communication system [18], [19], [20], [21] and compute a safe, optimal, and complete solution for navigation. However, they do not scale to large multi-robot systems with tens of robots. Many practical geometric decentralized methods for multi-agent systems are based on reciprocal velocity obstacles [7] or its variants [22]. These synthetic methods can be used during the training phase of learning algorithms.

B. Learning-Based Navigation Methods

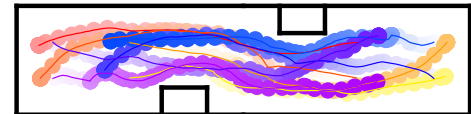
Learning-based collision avoidance techniques usually try to optimize a parameterized policy using the data collected from different tasks. Many navigation algorithms adopt a supervised learning paradigm to train collision avoidance policies. Muller et al. [23] present a vision-based static obstacle avoidance system using a 6-layer CNN to map input images to steering angles. Zhang et al. [24] describe a successor-feature-based deep reinforcement learning algorithm for robot navigation tasks based on raw sensory data. Barreto et al. [25] apply transfer learning to deploy a policy for new problem instances. Sergeant et al. [26] propose an approach based on multimodal deep autoencoders that enables a robot to learn how to navigate by observing a dataset of sensor inputs and motor commands collected while being tele-operated by a human. Ross et al. [27] adapt an imitation learning technique to train reactive heading policies based on the knowledge of a human pilot. Pfeiffer et al. [28] map the laser scan and goal positions to motion commands using expert demonstrations. To be effective, these methods need to collect training data in different environments, and the performance is limited by the quality of the training sets.

To overcome the limitations of supervised-learning, Tai et al. [29] present a mapless motion planner trained end-to-end without any manually designed features or prior demonstrations. Kahn et al. [30] propose an uncertainty-aware model-based learning algorithm that estimates the probability of collision, then uses that information to minimize the collisions at training time. To extend learning-based methods to highly dynamic environments, some decentralized techniques have been proposed. Godoy et al. [31] propose a Bayesian inference approach that computes a plan that minimizes the number of collisions while driving the robot to its goal. Chen et al. [14], [32] and Everett et al. [15] present multi-robot collision avoidance policies based on deep reinforcement learning, requiring the deployment of multiple sensors to estimate the state of nearby agents and moving obstacles. Yoon et al. [33] extend the framework of centralized training with

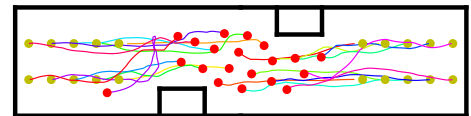
decentralized execution to perform additional optimization for inter-agent communication. Fan et al. [12] and Long et al. [16], [17] describe a decentralized multi-robot collision avoidance framework where each robot makes navigation decisions independently without any communication with other agents. It has been extended in terms of multiple sensors and explicit pedestrian motion prediction [34]. Other methods account for social constraints [15]. However, all these methods do not utilize global information about the robot or the environment, which could be used to improve the optimality of the resulting paths or handle challenging narrow scenarios.



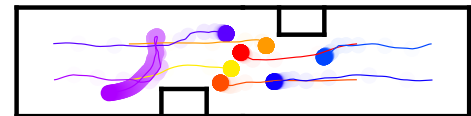
(a) Our algorithm is able to compute collision-free trajectories for 20 robots.



(b) We highlight some of the computed trajectories with temporal information computed using our algorithm.



(c) Trajectories by local learning method [16]. All agents do not reach the goal position.



(d) Selected trajectories by [16] with temporal information. The agents get stuck.

Fig. 2: Narrow Corridor: We compute the trajectories computed by DeepMNavigate and [16] for two groups of robots (20 total) exchanging their positions through narrow corridors. In (a) and (c), the yellow points correspond to the initial positions and the red points correspond to the final positions. (b) and (d), highlight the temporal information along the trajectories using color and transparency. Prior local planning methods [16] can only handle these scenarios with up to 12 agents and the geometric decentralized methods [7] cannot handle such cases. This benchmark is quite different from training datasets.

III. MULTI-ROBOT NAVIGATION

A. Problem Formulation and Notation

We consider the multi-robot navigation problem for non-holonomic differential drive robots. Our goal is to design a scheme that avoids collisions with obstacles and other robots and works well in dense and general environments. We describe the approach for 2D, but it can be extended to 3D workspaces and to robots with other dynamics constraints.

场景并拥有更多代理。这证明了我们方法的普遍性。

二.相关工作

A. Geometric Multi-Robot Navigation Algorithms

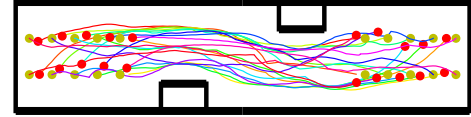
大多数现有算法都基于几何技术，例如基于采样的方法、几何优化或完整的运动规划算法。集中式方法假设每个机器人都可以基于某种全局数据结构或通信系统访问有关其他机器人状态的完整信息[18]、[19]、[20]、[21]，并计算安全、最优和完整的导航解决方案。然而，它们无法扩展到具有数十个机器人的大型多机器人系统。许多实用的多智能体系统几何分散方法都是基于相互速度障碍[7]或其变体[22]。这些合成方法可以在学习算法的训练阶段使用。

B. Learning-Based Navigation Methods

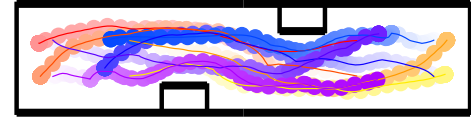
基于学习的碰撞避免技术通常尝试使用从不同任务收集的数据来优化参数化策略。许多导航算法采用监督学习范式来训练防撞策略。穆勒等人。[23]提出了一种基于视觉的静态避障系统，使用 6 层 CNN 将输入图像映射到转向角度。张等人。[24]描述了一种基于后继特征的深度强化学习算法，用于基于原始感官数据的机器人导航任务。巴雷托等人。[25]应用迁移学习来部署新问题实例的策略。中士等人。[26]提出了一种基于多模态深度自动编码器的方法，该方法使机器人能够通过观察人类远程操作时收集的传感器输入和运动命令的数据集来学习如何导航。罗斯等人。[27]采用模仿学习技术来训练基于人类飞行员知识的反应性航向策略。菲佛等人。[28]使用专家演示将激光扫描和目标位置映射到运动命令。为了有效，这些方法需要收集不同环境中的训练数据，并且性能受到训练集质量的限制。

为了克服监督学习的局限性，Tai 等人。[29]提出了一种经过端到端训练的无地图运动规划器，无需任何手动设计的功能或事先演示。卡恩等人。[30]提出了一种基于不确定性感知模型的学习算法，该算法估计碰撞概率，然后使用该信息来最小化训练时的碰撞。为了将基于学习的方法扩展到高度动态的环境，已经提出了一些分散的技术。戈多伊等人。[31]提出了一种贝叶斯推理方法，该方法可以计算一个计划，在驱动机器人到达目标的同时最大限度地减少碰撞次数。陈等人。[14]、[32]和 Everett 等人。[15]提出了基于深度强化学习的多机器人防撞策略，需要部署多个传感器来估计附近代理和移动障碍物的状态。尹等人。[33]扩大集中培训框架

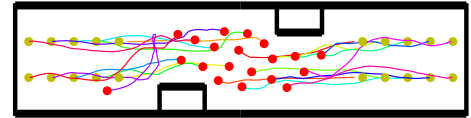
分散执行以对代理间通信进行额外的优化。范等人。[12]和朗等人。[16]、[17]描述了一种分散的多机器人防撞框架，其中每个机器人独立做出导航决策，无需与其他代理进行任何通信。它在多个传感器和显式行人运动预测方面得到了扩展[34]。其他方法考虑了社会约束[15]。然而，所有这些方法都没有利用有关机器人或环境的全局信息，这些信息可用于提高结果路径的最优性或处理具有挑战性的狭窄场景。



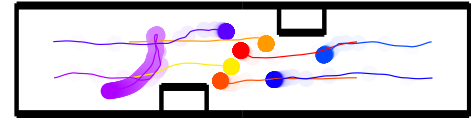
(a) Our algorithm is able to compute collision-free trajectories for 20 robots.



(b) We highlight some of the computed trajectories with temporal information computed using our algorithm.



(c) Trajectories by local learning method [16]. All agents do not reach the goal position.



(d) Selected trajectories by [16] with temporal information. The agents get stuck.

图 2: 狭窄走廊: 我们计算了由 DeepMNavigate 和 [16] 计算出的两组机器人 (总共 20 个) 通过狭窄走廊交换位置的轨迹。在 (a) 和 (c) 中，黄点对应于初始位置，红点对应于最终位置。(b) 和 (d)，使用颜色和透明度突出显示沿轨迹的时间信息。先前的局部规划方法 [16] 只能处理最多 12 个智能体的场景，而几何分散方法 [7] 无法处理此类情况。该基准与训练数据集有很大不同。

三. 多机器人导航

A. Problem Formulation and Notation

我们考虑非完整差分驱动机器人的多机器人导航问题。我们的目标是设计一种方案，避免与障碍物和其他机器人碰撞，并在密集和一般环境中运行良好。我们描述了 2D 方法，但它可以扩展到 3D 工作空间和具有其他动力学约束的机器人。

Let the number of robots be N_{rob} . We represent each robot as a disc with radius R . At each timestep t , the i -th robot ($1 \leq i \leq N_{\text{rob}}$) has access to an observation $\mathbf{o}_i^t$ and then computes an action $\mathbf{a}_i^t$ that drives the i -th robot towards its goal $\mathbf{g}_i^t$ from the current position $\mathbf{p}_i^t$. The observation of each robot includes four parts: $\mathbf{o}^t = [\mathbf{o}_z^t, \mathbf{o}_g^t, \mathbf{o}_v^t, \mathbf{o}_M^t]$, where $\mathbf{o}_z^t$ denotes the sensor measurement (e.g., laser sensor) of its surrounding environment, $\mathbf{o}_g^t$ stands for its relative goal position, $\mathbf{o}_v^t$ refers to its current velocity, and $\mathbf{o}_M^t$ is the robot motion information, which includes the global state of the system, discussed in Section IV. In this paper, we focus on analyzing and incorporating motion information in the navigation system. Meanwhile, there are N_{obs} static obstacles in the environment. We use $\mathbf{B}_k$ to denote the area occupied by a static k -th obstacle. The computed action $\mathbf{a}^t$ drives the robot to its goal while avoiding collisions with other robots and obstacles within the timestep Δt until the next observation $\mathbf{o}^{t+1}$ is received.

Let $\mathbb{L}$ be the set of trajectories for all robots, subject to the robot's kinematic constraints, i.e.:

$$\begin{aligned} \mathbb{L} = \{l_i, i = 1, \dots, N_{\text{rob}} | \mathbf{v}_i^t \sim \pi_\theta(\mathbf{a}_i^t | \mathbf{o}_i^t), \mathbf{p}_i^t = \mathbf{p}_i^{t-1} + \Delta t \cdot \mathbf{v}_i^t, \\ \forall j \in [1, N_{\text{rob}}], j \neq i, \|\mathbf{p}_i^t - \mathbf{p}_j^t\| > 2R \wedge \forall k \in [1, N_{\text{obs}}], \\ \forall \mathbf{q} \in \mathbf{B}_k, \|\mathbf{p}_i^t - \mathbf{q}\| > R \wedge \|\mathbf{v}_i^t\| \leq v_i^{\text{max}}\}, \end{aligned} \quad (1)$$

where $\mathbf{p}_i^0$ is the initial position of the robot and $\mathbf{p}_i^t$ are the positions at timestep t . $\mathbf{v}_i^t$ is the current linear velocity on the 2D plane (i.e. v_x, v_y) as a result of the action $\mathbf{a}_i^t$. The agent we are simulating is non-holonomic and can only control the linear velocity on the X axis and the angular velocity on the Z axis (which is used to describe the rotation in the 2D plane).

B. Multi-Agent Navigation Using Reinforcement Learning

Our approach builds on prior reinforcement learning approaches that use local information comprised of various observations. Some of them only utilize three of the four elements mentioned in III-A. The term $\mathbf{o}_z^t$ may include the measurements of the last three consecutive frames from a sensor. The relative goal position $\mathbf{o}_g^t$ in these cases is a 2D vector representing the goal position in polar coordinates with respect to the robots current position. The observed velocity $\mathbf{o}_v^t$ includes the current velocity of the robot. These observations are normalized using the statistics aggregated during training[35], [36]. This normalization can make RL training more stable and improve its performance. The action of a differential robot includes the translational and rotational velocity, i.e. $\mathbf{a}^t = [v^t, \omega^t]$, $v \in (0, 1)$, $\omega \in (-1, 1)$. We use the following reward function to guide a team of robots:

$$r_i^t = (g_r)_i^t + (c_r)_i^t + (\omega_r)_i^t. \quad (2)$$

When the robot gets closer or reaches its goal, it is rewarded as

$$(g_r)_i^t = \begin{cases} r_{\text{arrival}} & \text{if } \|\mathbf{p}_i^t - \mathbf{g}_i\| < 0.1 \\ r_{\text{approaching}}(\|\mathbf{p}_i^{t-1} - \mathbf{g}_i\| - \|\mathbf{p}_i^t - \mathbf{g}_i\|) & \text{otherwise.} \end{cases} \quad (3)$$

The $\|\mathbf{p}_i - \mathbf{g}_i\|$ item denotes the distance between the robot and its goal. When there is a collision, it is penalized using

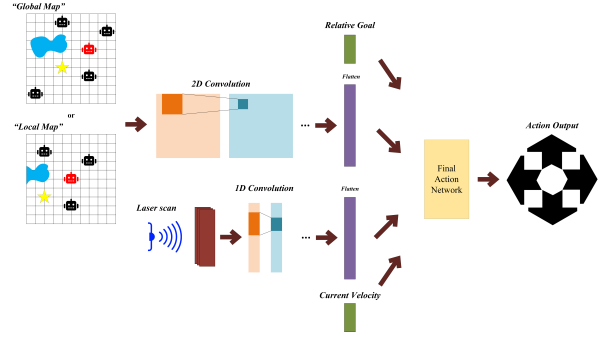


Fig. 3: We highlight the architecture of our policy network (DeepM-Navigate), including (*global* and *local*) maps used by our approach. The *global map* is based on the world coordinate system and each *local map* is centered at the corresponding robot's current location. The red robot represents the map's corresponding robot, black robots represent the neighboring robots, the yellow star represents the goal, and the blue area is an obstacle. In our implementation, the map is discretized and assigned different values. We use a 2D convolutional neural network to handle the additional global information input from the map and fully-connected network to compute the action for each robot.

the function

$$(c_r)_i^t = \begin{cases} r_{\text{collision}} & \text{if } \|\mathbf{p}_i^t - \mathbf{p}_j^t\| < 2R \\ & \text{or } \|\mathbf{p}_i^t - \mathbf{q}\| < R, \mathbf{q} \in \mathbf{B}_k \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

In addition to collision avoidance, one of our goals is generating a smooth path. A simple technique is to impose penalties whenever there are large rotational velocities. Although it is not a standard way to obtain smooth path, we found this technique can achieve a smooth trajectory empirically. This can be expressed as

$$(\omega_r)_i^t = \begin{cases} r_{\text{smooth}}|\omega_i^t| & \text{if } |\omega_i^t| > 0.7. \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

where r_{arrival} , $r_{\text{approaching}}$, $r_{\text{collision}}$ and r_{smooth} are parameters used to control the reward. These parameters provide reward feedback for the agents, which makes the training process more stable [37]. In practice, the reward parameters can be tuned to obtain desirable behaviors (e.g., learn more conservative behaviors without a larger collision penalty). We do not change the reward function when we use our approach for different environments.

IV. DEEPMNAVIGATE: TRAJECTORY COMPUTATION USING GLOBAL INFORMATION

In this section, we present our novel, learning-based, multi-agent navigation algorithm that uses positional information of other agents. Our formulation is based on motion information maps and uses a 3-layer CNN to generate a suitable action for each agent.

A. Motion Information Maps

Prior rule-based decentralized methods such as [7] use information corresponding to the position and velocity of each agent to compute a locally-optimal and collision-free trajectory. Our goal is to compute similar state information to design better learning-based navigation algorithms. Such state information can be either gathered based on some communication with nearby robots or computed using a deep

设机器人数量为 N_{Robo} 。我们将每个机器人表示为半径为 R 的圆盘。在每个时间步 t ，第 i 个机器人 ($1 \leq i \leq N_{\text{Robo}}$) 都可以访问观测值 $\mathbf{o}_i^t$ ，然后计算一个动作 $\mathbf{a}_i^t$ ，驱动第 i 个机器人从当前位置 $\mathbf{p}_i^t$ 朝着目标 $\mathbf{g}_i^t$ 前进。每个机器人的观测包括四个部分： $\mathbf{o}^t = [\mathbf{o}_z^t, \mathbf{o}_g^t, \mathbf{o}_v^t, \mathbf{o}_M^t]$ ，其中 $\mathbf{o}_z^t$ 表示其周围环境的传感器测量（例如激光传感器）， $\mathbf{o}_g^t$ 表示其相对目标位置， $\mathbf{o}_v^t$ 表示其当前速度， $\mathbf{o}_M^t$ 是机器人运动信息，其中包括系统的全局状态，这将在第四节中讨论。在本文中，我们重点分析导航系统中的运动信息并将其纳入其中。同时，环境中存在 N_{Obs} 静态障碍物。我们使用 $\mathbf{B}_k$ 表示第 k 个静态障碍物占据的区域。计算出的动作 $\mathbf{a}^t$ 驱动机器人到达目标，同时避免在时间步长 Δt 内与其他机器人和障碍物发生碰撞，直到收到下一个观察值 $\mathbf{o}^{t+1}$ 。

令 $\mathbb{L}$ 为所有机器人的轨迹集，受机器人运动学约束，即：

$$\begin{aligned} \mathbb{L} = \{l_i, i = 1, \dots, N_{\text{Robo}} | \mathbf{v}_i^t \sim \pi_\theta(\mathbf{a}_i^t | \mathbf{o}_i^t), \mathbf{p}_i^t = \mathbf{p}_i^{t-1} + \Delta t \cdot \mathbf{v}_i^t, \\ \forall j \in [1, N_{\text{Robo}}], j \neq i, \|\mathbf{p}_i^t - \mathbf{p}_j^t\| > 2R \wedge \forall k \in [1, N_{\text{Obs}}], \\ \forall \mathbf{q} \in \mathbf{B}_k, \|\mathbf{p}_i^t - \mathbf{q}\| > R \wedge \|\mathbf{v}_i^t\| \leq v_i^{\max}\}, \end{aligned} \quad (1)$$

其中 $\mathbf{p}_i^0$ 是机器人的初始位置， $\mathbf{p}_i^t$ 是时间步 t 的位置。

$\mathbf{v}_i^t$ 是由于动作 $\mathbf{a}_i^t$ 而导致的 2D 平面（即 v_x, v_y ）上的当前线速度。我们模拟的智能体是非完整的，只能控制 X 轴上的线速度和 Z 轴上的角速度（用于描述 2D 平面中的旋转）。

B. Multi-Agent Navigation Using Reinforcement Learning

我们的方法建立在先前的强化学习方法的基础上，该方法使用由各种观察组成的本地信息。其中一些仅利用 I II-A 中提到的四种元素中的三种。术语 $\mathbf{o}_z^t$ 可以包括来自传感器的最后三个连续帧的测量值。在这些情况下，相对目标位置 $\mathbf{o}_g^t$ 是一个 2D 矢量，表示极坐标中相对于机器人当前位置的目标位置。观测到的速度 $\mathbf{o}_v^t$ 包括机器人的当前速度。使用训练期间汇总的统计数据对这些观察结果进行归一化[35]、[36]。这种归一化可以使 RL 训练更加稳定并提高其性能。差动机器人的动作包括平移速度和旋转速度，即 $\mathbf{a}^t = [v^t, \omega^t]$ ， $v \in (0, 1)$ ， $\omega \in (-1, 1)$ 。我们使用以下奖励函数来引导机器人团队：

$$r_i^t = (g_r)_i^t + (c_r)_i^t + (\omega_r)_i^t. \quad (2)$$

当机器人接近或达到目标时，它会获得奖励：

$$(g_r)_i^t = \begin{cases} r_{\text{arrival}} & \text{if } \|\mathbf{p}_i^t - \mathbf{g}_i\| < 0.1 \\ r_{\text{approaching}}(\|\mathbf{p}_i^{t-1} - \mathbf{g}_i\| - \|\mathbf{p}_i^t - \mathbf{g}_i\|) & \text{otherwise.} \end{cases} \quad (3)$$

$\|\mathbf{p}_i - \mathbf{g}_i\|$ 项表示机器人与其目标之间的距离。当发生碰撞时，将受到处罚

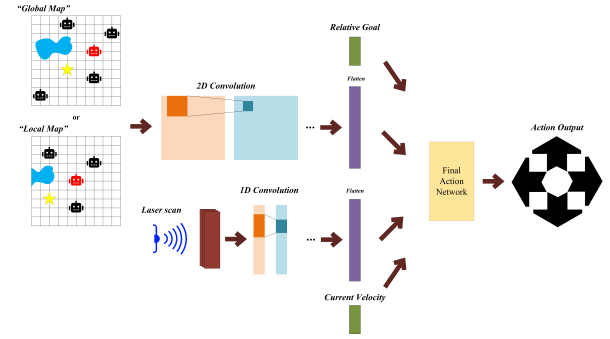


图 3: 我们重点介绍了我们的策略网络 (DeepM-Navigate) 的架构，包括我们的方法使用的 (*global* 和 *local*) 地图。 *global map* 基于世界坐标系，每个 *local map* 以相应机器人的当前位置为中心。红色机器人代表地图对应的机器人，黑色机器人代表邻近的机器人，黄色星星代表目标，蓝色区域为障碍物。在我们的实现中，地图被离散化并分配了不同的值。我们使用 2D 卷积神经网络来处理来自地图的额外全局信息输入，并使用全连接网络来计算每个机器人的动作。

乐趣 ction

$$(c_r)_i^t = \begin{cases} r_{\text{collision}} & \text{if } \|\mathbf{p}_i^t - \mathbf{p}_j^t\| < 2R \\ & \text{or } \|\mathbf{p}_i^t - \mathbf{q}\| < R, \mathbf{q} \in \mathbf{B}_k \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

除了避免碰撞之外，我们的目标之一是生成一条平滑的路径。一种简单的技术是每当旋转速度很大时就施加惩罚。尽管这不是获得平滑路径的标准方法，但我们发现该技术可以根据经验实现平滑轨迹。这可以表示为

$$(\omega_r)_i^t = \begin{cases} r_{\text{smooth}}|\omega_i^t| & \text{if } |\omega_i^t| > 0.7. \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

其中 r_{arrival} 、 $r_{\text{approaching}}$ 、 $r_{\text{collision}}$ 和 r_{smooth} 是用于控制奖励的参数。这些参数为智能体提供奖励反馈，这使得训练过程更加稳定[37]。在实践中，可以调整奖励参数以获得所需的行为（例如，学习更保守的行为而不需要更大的碰撞惩罚）。当我们针对不同环境使用我们的方法时，我们不会改变奖励函数。

四. DEEPMNAVIGATE: 使用全局信息进行轨迹计算

在本节中，我们将介绍新颖的、基于学习的多智能体导航算法，该算法使用其他智能体的位置信息。我们的公式基于运动信息图，并使用 3 层 CNN 为每个代理生成合适的动作。

A. Motion Information Maps

先前基于规则的分散方法（例如[7]）使用与每个代理的位置和速度相对应的信息来计算局部最优且无碰撞的轨迹。我们的目标是计算类似的状态信息，以设计更好的基于学习的导航算法。此类状态信息可以根据与附近机器人的某些通信来收集，也可以使用深度学习来计算

network that uses raw sensor data. In our formulation, we use maps that consist of each agent’s location as input. In particular, we use two different map representations: one corresponds to all the robots based on the world coordinate system and is called the *global-map*; the other map is centered at each robot’s current location and uses the relative coordinate system and is called the *local-map*.

We use the following method to compute the global-map and the local-map. During each timestep t , we specify the i -th robot’s position in the world frame as $\mathbf{x}_i^t \in \mathbb{R}^2$. We also use the goal positions $\mathbf{g}_i, \forall 1 \leq i \leq N_{\text{rob}}$, obstacle information $\mathbf{B}_k, \forall 1 \leq k \leq N_{\text{obs}}$, to build the map $M_i^t \in \mathbb{R}^{h \times w}$ for the i -th robot. Assume the size of the simulated robot scenario is $H \times W$, where H represents the height, W represents the width, and the origin of the world frame is located at $(\frac{H}{2}, \frac{W}{2})$. Each pixel of the map ($M_i^t(p, q), \forall 1 \leq p \leq h, 1 \leq q \leq w$) indicates which kind of object lies in the small area $\mathcal{A}_{pq} = \left(\frac{(p-1)H}{h} - \frac{H}{2}, \frac{pH}{h} - \frac{H}{2} \right] \times \left(\frac{(q-1)W}{w} - \frac{W}{2}, \frac{qW}{w} - \frac{W}{2} \right]$ in the world frame. Assuming each object’s radius is r_i , then $M_i^t(p, q)$ corresponds to:

$$M_i^t(p, q) = \begin{cases} 1, & \{y | \|y - \mathbf{x}_i^t\| \leq R\} \cup \mathcal{A}_{pq} \neq \emptyset \\ 2, & \exists 1 \leq j \leq N_{\text{rob}}, j \neq i, \\ & \text{s.t. } \{y | \|y - \mathbf{x}_j^t\| \leq R\} \cup \mathcal{A}_{pq} \neq \emptyset \\ 3, & \{y | \|y - \mathbf{g}_i\| \leq R\} \cup \mathcal{A}_{pq} \neq \emptyset \\ 4, & \exists 1 \leq k \leq N_{\text{obs}}, \text{s.t. } \mathbf{B}_k \cup \mathcal{A}_{pq} \neq \emptyset \\ 0, & \text{otherwise,} \end{cases} \quad (6)$$

where “1” represents the corresponding robot, “2” represents the neighboring robots, “3” represents the robot’s goal, “4” represents the obstacles and “0” represents the empty background (i.e. free space). This is highlighted in Fig. 3.

In some scenarios, there could be a restriction on the robot’s movement in terms of static obstacles or regions that are not accessible. Our global-map computation takes this into account in terms of representing $M_i^t(p, q)$. However, these maps may not capture scenes with no clear boundaries or very large environments spread over a large area. If we use a *global-map* with the world coordinate representation for all the agents, the resulting map would be extremely large and would involve a high computational cost and memory overhead. In these cases, we use the *local-map* for each agent, instead of considering the whole scenario, which would have a size $H \times W$. These local maps only account for information in a relatively small neighborhood with a fixed size $H_l \times W_l$. The size of the local neighborhood (H_l, W_l) can be tuned to find a better performance for different applications. In addition to the position information, these maps may contain other state information of the robot, including velocity, orientation, or dynamics constraints as additional channels.

B. Proximal Policy Optimization

We use proximal policy optimization [38] to optimize the overall system. This training algorithm has the stability and reliability of trust-region methods: i.e., it tries to compute an update at each step that minimizes the cost function while ensuring that the deviation from the previous policy

is relatively small. The resulting proximal policy algorithm updates the network using all the steps after several continuous simulations (i.e. after each robot in the entire system reaches its goal or stops running due to collisions) instead of using only one step to ensure the stability of network optimization. In these cases, if we store the robot positions or motion information as dense matrices corresponding to the formulation in Eq. 6, it would require a significant amount of memory and also increase the overall training time. Instead, we use a sparse matrix representation for M_i^t . We compute the non-zero entries of M_i^t based on the current position of each robot, the goal positions and obstacle information using Eq. 6. To feed the input to our neural network, we generate dense representations using temporary sparse storage. This design choice allows us to train the system using trajectories from 58 agents performing 450 actions using only 2.5GB memory. More details on the training step are given in Sec. V.

C. Network

To analyze a large matrix and produce a low-dimensional feature for input M_i^t , we use a convolutional neural network (CNN) because such network structures are useful for handling an image-like input (or our map representation). Our network has three convolutional layers with architecture, as shown in Fig. 3. Our network extracts the related locations of different objects in the environment and could guide the overall planner to avoid other agents and obstacles to reach the goal.

Our approach to handling raw sensor data (e.g., 2D laser scanner data) uses the same structure as local methods[16], i.e. a two-layer, 1D convolutional network. Overall, we use a two-layer, fully-connected network that takes as input the observation features, including the feature generated by the 1D & 2D CNNs, related goal position, and observed velocity. This generates the action output and local path for each robot. We highlight the whole pipeline of the network for local and global information in Fig. 3 and Algorithm 1.

Algorithm 1 Policy Making for DeepMNavigate

```

1: for timestep  $t = 1, 2, \dots$  do
2:   // Each robot runs individually in parallel
3:   for robot  $i = 1, 2, \dots, N$  do
4:     Collect observation  $\mathbf{o}_i^t = [\mathbf{o}_z^t, \mathbf{o}_g^t, \mathbf{o}_v^t, \mathbf{o}_M^t]$ 
5:     Run policy  $\pi_\theta$  represented by the network, get action  $\mathbf{a}_i^t$ 
6:     Update the robot position  $\mathbf{p}_i^t$  according to action  $\mathbf{a}_i^t$ 
7:   end for
8: end for
```

D. Network Training

Our training strategy extends the method used by learning algorithms based on local information [16], [14]. To accelerate the training process, we divide the overall training computation into two stages. In the first stage, we use k robots (e.g. $k=20$) with random initial positions and random goals in a fully-free environment. In the second stage, we include

使用原始传感器数据的网络。在我们的公式中，我们使用包含每个代理位置的地图作为输入。特别地，我们使用两种不同的地图表示：一种对应于基于世界坐标系的所有机器人，称为 $global-map$ ；另一种对应于基于世界坐标系的所有机器人，称为 $global-map$ 。另一张地图以每个机器人的当前位置为中心，使用相对坐标系，称为 $local-map$ 。

我们使用以下方法来计算全局映射和局部映射。在每个时间步 t 期间，我们将第 i 个机器人在世界坐标系中的位置指定为 $\mathbf{x}_i^t \in \mathbb{R}^2$ 。我们还使用目标位置 $\mathbf{g}_i, \forall 1 \leq i \leq N_{\text{rob}}$ 、障碍物信息 $\mathbf{B}_k, \forall 1 \leq k \leq N_{\text{obs}}$ 为第 i 个机器人构建地图 $M_i^t \in \mathbb{R}^{h \times w}$ 。假设模拟机器人场景的大小为 $H \times W$ ，其中 H 表示高度， W 表示宽度，世界坐标系的原点位于 $(\frac{H}{2}, \frac{W}{2})$ 。地图的每个像素 $(M_i^t(p, q), \forall 1 \leq p \leq h, 1 \leq q \leq w)$ 指示哪种对象位于世界坐标系中的小区域 $\mathcal{A}_{pq} = \left(\left(\frac{(p-1)H}{h} - \frac{H}{2}, \frac{pH}{h} - \frac{H}{2} \right) \times \left(\frac{(q-1)W}{w} - \frac{W}{2}, \frac{qW}{w} - \frac{W}{2} \right) \right)$ 中。假设每个物体的半径为 r_i ，则 $M_i^t(p, q)$ 对应：

$$M_i^t(p, q) = \begin{cases} 1, & \{y \mid \|y - \mathbf{x}_i^t\| \leq R\} \cup \mathcal{A}_{pq} \neq \emptyset \\ 2 & \exists 1 \leq j \leq N_{\text{rob}}, j \neq i, \\ & \text{s.t. } \{y \mid \|y - \mathbf{x}_j^t\| \leq R\} \cup \mathcal{A}_{pq} \neq \emptyset \\ 3, & \{y \mid \|y - \mathbf{g}_i\| \leq R\} \cup \mathcal{A}_{pq} \neq \emptyset \\ 4, & \exists 1 \leq k \leq N_{\text{obs}}, \text{s.t. } \mathbf{B}_k \cup \mathcal{A}_{pq} \neq \emptyset \\ 0, & \text{otherwise,} \end{cases} \quad (6)$$

其中“1”代表相应的机器人，“2”代表邻近的机器人，“3”代表机器人的目标，“4”代表障碍物，“0”代表空背景（即自由空间）。这在图 3 中突出显示。

在某些情况下，机器人的运动可能会因静态障碍物或无法到达的区域而受到限制。我们的全局地图计算在表示 $M_i^t(p, q)$ 方面考虑了这一点。然而，这些地图可能无法捕捉没有明确边界的场景或分布在大面积的非常大的环境。如果我们使用具有世界坐标表示的 $global-map$ 来表示所有代理，则生成的地图将非常大，并且会涉及很高的计算成本和内存开销。在这些情况下，我们对每个代理使用 $local-map$ ，而不是考虑大小为 $H \times W$ 的整个场景。这些局部地图仅考虑具有固定大小 $H_l \times W_l$ 的相对较小邻域中的信息。可以调整局部邻域 (H_l, W_l) 的大小，以便为不同的应用程序找到更好的性能。除了位置信息之外，这些地图还可能包含机器人的其他状态信息，包括速度、方向或动态约束作为附加通道。

B. Proximal Policy Optimization

我们使用近端策略优化[38]来优化整个系统。该训练算法具有信任域方法的稳定性和可靠性：即，它尝试在每一步计算更新，以最小化成本函数，同时确保与先前策略的偏差

相对较小。由此产生的近端策略算法在多次连续模拟后（即整个系统中的每个机器人达到目标或因碰撞而停止运行后）使用所有步骤更新网络，而不是仅使用一个步骤，以确保网络优化的稳定性。在这些情况下，如果我们将机器人位置或运动信息存储为与等式1中的公式相对应的密集矩阵。6、它需要大量的内存，并且还会增加总体训练时间。相反，我们对 M_i^t 使用稀疏矩阵表示。我们根据每个机器人的当前位置、目标位置和障碍物信息，使用等式计算 M_i^t 的非零条目。6. 为了将输入提供给我们神经网络，我们使用临时稀疏存储生成密集表示。这种设计选择使我们能够使用 58 个智能体的轨迹来训练系统，这些智能体仅使用 2.5GB 内存执行 450 个动作。有关训练步骤的更多详细信息在第 2 节中给出。五、

C. Network

为了分析大矩阵并为输入 M_i^t 生成低维特征，我们使用卷积神经网络（CNN），因为这种网络结构对于处理类似图像的输入（或我们的地图表示）很有用。我们的网络具有三个卷积层的架构，如图3所示。我们的网络提取环境中不同对象的相关位置，可以指导整体规划器避开其他代理和障碍物以达到目标。

我们处理原始传感器数据（例如，2D 激光扫描仪数据）的方法使用与本地方法相同的结构[16]，即两层一维卷积网络。总体而言，我们使用一个两层全连接网络，将观察特征作为输入，包括 1D 和 2D CNN 生成的特征、相关目标位置和观察到的速度。这会生成每个机器人的动作输出和本地路径。我们在图 3 和算法 1 中强调了本地和全局信息网络的整个管道。

Algorithm 1 Policy Making for DeepMNavigate

```

1: for timestep  $t = 1, 2, \dots$  do
2:   // Each robot runs individually in parallel
3:   for robot  $i = 1, 2, \dots, N$  do
4:     Collect observation  $\mathbf{o}_i^t = [\mathbf{o}_z^t, \mathbf{o}_g^t, \mathbf{o}_v^t, \mathbf{o}_M^t]$ 
5:     Run policy  $\pi_\theta$  represented by the network, get action  $\mathbf{a}_i^t$ 
6:     Update the robot position  $\mathbf{p}_i^t$  according to action  $\mathbf{a}_i^t$ 
7:   end for
8: end for

```

D. Network Training

我们的训练策略扩展了基于局部信息的学习算法所使用的方法[16], [14]。为了加速训练过程，我们将整个训练计算分为两个阶段。在第一阶段，我们在完全自由的环境中具有随机初始位置和随机目标的 k 机器人（例如 $k=20$ ）。在第二阶段，我们包括

Layer	Convolutional Filter	Stride	Padding	Activation Fuction	Output Size
Input	-	-	-	-	$250 \times 250 \times 1$
Conv 1	$7 \times 7 \times 1 \times 8$	1×1	'SAME'	ReLU	$250 \times 250 \times 8$
Max Pooling 1	3×3	2×2	'SAME'	-	$125 \times 125 \times 8$
Conv 2	$7 \times 7 \times 8 \times 12$	1×1	'SAME'	ReLU	$125 \times 125 \times 12$
Max Pooling 2	3×3	2×2	'SAME'	-	$63 \times 63 \times 12$
Conv 3	$7 \times 7 \times 12 \times 20$	1×1	'SAME'	ReLU	$63 \times 63 \times 20$
Max Pooling 3	3×3	2×2	'SAME'	-	$32 \times 32 \times 20$
Flatten	-	-	-	-	20480
Fully connected	20480×128	-	-	ReLU	384
Fully connected	128×64	-	-	ReLU	256

TABLE I: Architecture and hyper-parameters of a convolutional neural network that considers the global information.

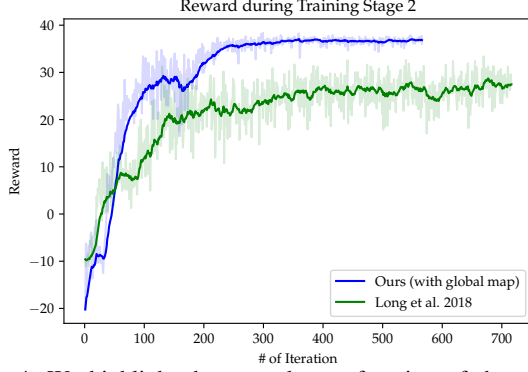


Fig. 4: We highlight the reward as a function of the number of iterations during the second stage of the overall training algorithm. We compare the performance of a reinforcement learning algorithm that only uses local information [16] to our method, which uses local and global information. Our method obtains a higher reward than [16] due to the global information.

a more challenging environments, such as a narrow passage, random obstacles, etc. We show the training scenarios in Fig. 5. These varying training environments and the large number of robots in the system can result in a good overall policy. Moreover, the use of global information results in much larger network parameters. We use a 20480×128 FC layer, which is more difficult to train than a relatively small, simple network that only accounts for local information. To accelerate the training process and generate accurate results, we do not train the entire network from scratch and instead include pre-training. During the first stage, we retrain the network with additional structures corresponding to the global information (i.e. the global-map) using the pre-trained local information network part. This pre-training stage uses the parameters proposed in [16]. We highlight the reward as a function of the iterations during the second stage of the training and compare the overall reward computation with that described in the local method [16] in Fig. 4. Notice that, since we use a 2D convolutional neural network, our overall training algorithm needs more time during each iteration of the training (ours around 1200s vs [16] around 400s). As a result, we do not perform the same number of training iterations as [16], as shown in Fig. 4. The total training time is around 40 hours.

V. IMPLEMENTATION AND PERFORMANCE

In this section, we discuss the performance of our multi-agent navigation algorithm (DeepMNavigate) on complex scenarios and highlight the benefits over prior reinforcement learning methods that only use local information [16].

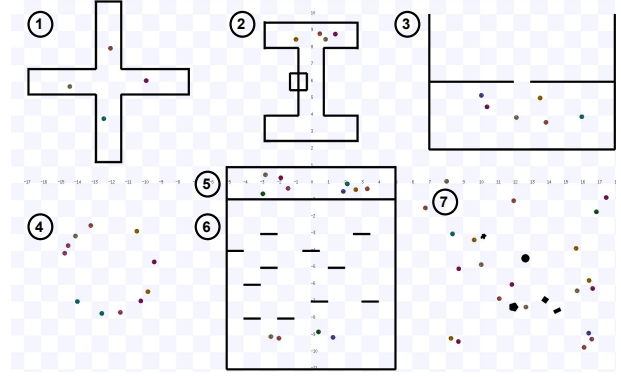


Fig. 5: Scenarios corresponding to challenging environments used in the second stage of training. This two-stage training improves the performance

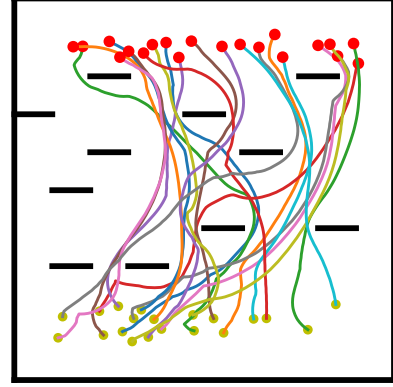


Fig. 6: **Room with Obstacles:** We highlight the trajectories of 20 robots in a room with multiple obstacles. We highlight the initial position of the agent (yellow) and the final position (red) along with multiple obstacles. Prior learning methods that only use local methods [16], [14] will take more time and may not be able to handle such scenarios when the number of obstacles or the number of agents increases.

A. Parameters

During simulation, the radius is set as $R = 0.12m$. In the current implementation, we set $H = W = 500m$ for the global-map and $H_l = W_l = 250m$ for each local-map. In both situations, we set $w = h = 250$. Although a larger map (e.g., $w = h = 500$) could include more details from the system, it would significantly increase the network size and the final running time. For instance, the memory requirement of CNN increases quadratically with the input size. In current implementation, we use a PC with 32-core CPU, 32GB memory and one NVIDIA RTX 2080 Ti. To include the additional global information, the algorithm consumes 1.63GB CPU memory, 970MB GPU memory and requires 0.25s for computing one time step, compared to [16] as 1.57 GB CPU memory, 340MB GPU memory and 0.2s. The overhead is not significant. For parameters in the reward function, we set $r_{arrival} = 15$, $r_{collision} = -15$, $r_{approaching} = 2.5$, and $r_{smooth} = -0.1$. We choose $r_{arrival}$ and $r_{collision}$ of the same magnitude to obtain an effective, safe behavior, which is a key metric to evaluate robots' trajectories; $r_{approaching}$ is chosen to encourage robots to approach their goal as fast as they can, which provides a dense feedback to make convergence relatively faster; a small

Layer	Convolutional Filter	Stride	Padding	Activation Fuction	Output Size
Input	-	-	-	-	$250 \times 250 \times 1$
Conv 1	$7 \times 7 \times 1 \times 8$	1×1	'SAME'	ReLU	$250 \times 250 \times 8$
Max Pooling 1	3×3	2×2	'SAME'	-	$125 \times 125 \times 8$
Conv 2	$7 \times 7 \times 8 \times 12$	1×1	'SAME'	ReLU	$125 \times 125 \times 12$
Max Pooling 2	3×3	2×2	'SAME'	-	$63 \times 63 \times 12$
Conv 3	$7 \times 7 \times 12 \times 20$	1×1	'SAME'	ReLU	$63 \times 63 \times 20$
Max Pooling 3	3×3	2×2	'SAME'	-	$32 \times 32 \times 20$
Flatten	-	-	-	-	20480
Fully connected	20480×128	-	-	ReLU	384
Fully connected	128×64	-	-	ReLU	256

表一：考虑全局信息的卷积神经网络的架构和超参数。

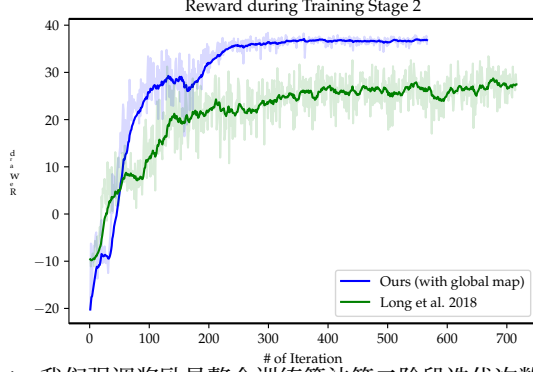


图 4：我们强调奖励是整个训练算法第二阶段迭代次数的函数。我们将仅使用局部信息 [16] 的强化学习算法的性能与我们使用局部和全局信息的方法进行比较。由于全局信息，我们的方法获得了比[16]更高的奖励。

更具挑战性的环境，例如狭窄的通道、随机障碍物等。我们在图 5 中显示了训练场景。这些不同的训练环境和系统中的大量机器人可以产生良好的整体策略。此外，全局信息的使用会导致更大的网络参数。我们使用 20480×128 FC 层，它比仅考虑本地信息的相对较小、简单的网络更难训练。为了加速训练过程并生成准确的结果，我们不会从头开始训练整个网络，而是进行预训练。在第一阶段，我们使用预先训练的局部信息网络部分，使用与全局信息（即全局地图）相对应的附加结构来重新训练网络。该预训练阶段使用[16]中提出的参数。我们强调奖励是训练第二阶段迭代的函数，并将整体奖励计算与图 4 中局部方法 [16] 中描述的计算进行比较。请注意，由于我们使用 2D 卷积神经网络，我们的整体训练算法在训练的每次迭代期间需要更多时间（我们的大约 1200 秒，[16] 大约 400 秒）。因此，我们没有执行与[16]相同数量的训练迭代，如图4所示。总训练时间约为40小时。

五、实施与绩效

在本节中，我们讨论多智能体导航算法（DeepMNavgate）在复杂场景中的性能，并强调相对于先前仅使用局部信息的强化学习方法的优势[16]。

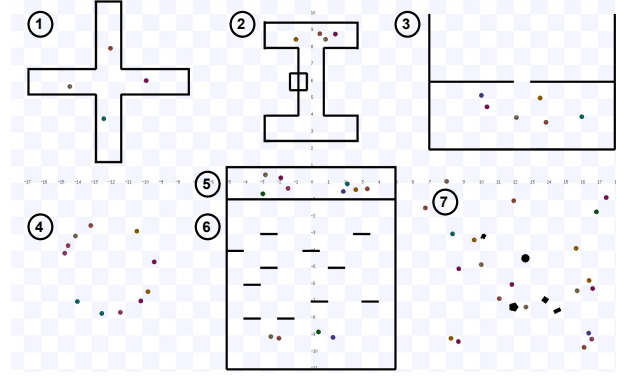


图 5：与第二阶段训练中使用的挑战性环境相对应的场景。这种两阶段训练提高了表现

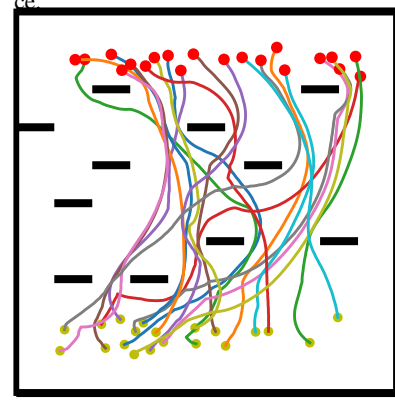


图 6：有障碍物的房间：我们突出显示了 20 个机器人在有多个障碍物的房间中的轨迹。我们突出显示代理的初始位置（黄色）和最终位置（红色）以及多个障碍物。先前仅使用局部方法的学习方法[16]、[14]将花费更多时间，并且当障碍物数量或代理数量增加时可能无法处理此类场景。

A. Parameters

仿真时，半径设置为 $R = 0.12m$ 。在当前的实现中，我们为全局映射设置 $H = W = 500m$ ，为每个本地映射设置 $H_l = W_l = 250m$ 。在这两种情况下，我们都设置 $w = h = 250$ 。虽然更大的地图（例如， $w = h = 500$ ）可以包含来自系统的更多详细信息，但它会显着增加网络大小和最终运行时间。例如，CNN 的内存需求随着输入大小呈二次方增加。在当前的实施中，我们使用一台具有 32 核 CPU、32GB 内存和一个 NVIDIA RTX 2080 Ti 的 PC。为了包含额外全局信息，该算法消耗 1.63GB CPU 内存、970MB GPU 内存，计算一个时间步需要 0.25 秒，而[16]为 1.57 GB CPU 内存、340 MB GPU 内存和 0.2 秒。开销并不大。对于奖励函数中的参数，我们设置 $r_{arrival} = 15$ 、 $r_{collision} = -15$ 、 $r_{approaching} = 2.5$ 和 $r_{smooth} = -0.1$ 。我们选择相同量级的 $r_{arrival}$ 和 $r_{collision}$ 来获得有效、安全的行为，这是评估机器人轨迹的关键指标；选择 $r_{approaching}$ 是为了鼓励机器人尽快接近目标，这提供了密集的反馈，使收敛速度相对更快；一个小

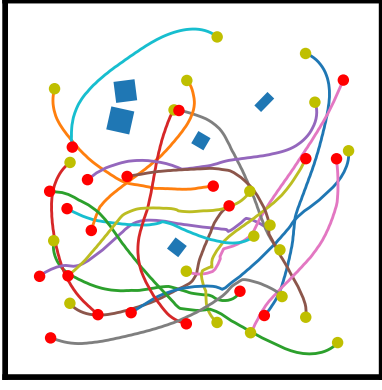


Fig. 7: **Random Start and Goal Positions:** Simulated trajectories of 20 robots from and to random positions in a scene with obstacles. The yellow points are the initial positions and the red points are the final positions. The blue areas are obstacles with random locations and orientations.

r_{smooth} value should regularize the trajectory and make it smoother.

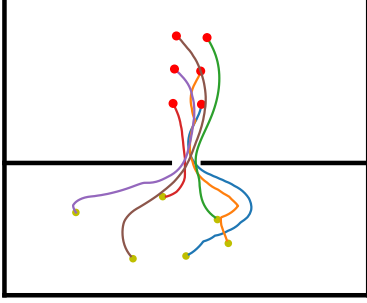


Fig. 8: **Room Evacuation:** Simulated trajectories of 6 robots evacuating a room with our algorithm. The yellow points are the initial positions and the red points are the final positions. Even with a narrow passage, our DeepMNavigate algorithm works well. This benchmark is quite different from training data.

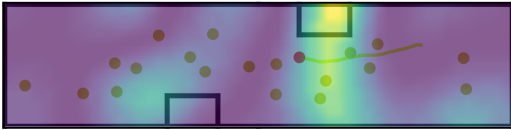


Fig. 9: Perturbation saliency generated by method [39] in the narrow corridor benchmark. Red point represents the current agent and yellow points represent the surrounding agents. Yellow area is saliency for the action policy.

B. Evaluation Metrics and Benchmarks

To evaluate the performance of our navigation algorithm, we use the following metrics:

- *Success rate:* the ratio of the number of robots reaching their goals in a certain time limit without any collisions to the total number of robots in the environment.
- *Collision or stuck rate:* if the robot cannot reach the destination within a limited time or collides, they are considered as getting stuck or in-collisions, respectively.

- *Extra time:* the difference between the average travel time of all robots and the lower bound of the robots' travel time. The latter is computed as the average travel time when going straight towards the goal at the maximum speed without checking for any collisions.
- *Average speed:* the average speed of all robots during the navigation.

We have evaluated our algorithm in five challenging and representative benchmarks:

- *Circle Crossing:* The robots are uniformly placed around a circle and the goal position of each robot is on the opposite end of the circle based on the diameter. The scenarios are widely used in prior multi-agent navigation algorithms [7], [16] (Fig. 1).
- *Narrow Corridor:* Two groups of robots exchange their positions through a narrow corridor. This benchmark is hard for geometric decentralized methods [7], which cannot navigate robots through narrow passages (Fig. 2).
- *Room with Obstacles:* The robots cross across a room full of obstacles, from one side to the other. Methods only using local information [16] will spend more time finding the path to the goal and may even fail due to lack of global information (Fig. 6).
- *Random Starts and Goals:* The robots start from random initial positions and moves to random goal positions. Also, there are obstacles with random locations and orientation. Global information will help find safer and faster paths (Fig. 7).
- *Room Evacuation:* The robots start from random initial positions and evacuate to the outside of the room. They need to cross one small door while avoiding collisions (Fig. 8).

C. New and Different Benchmarks

We have evaluated the performance of our method on benchmarks that are quite different from the training data in terms of layout and the inclusion of narrow passages. They also use different numbers of agents. In the Circle Crossing benchmark, we only train with 12 agents, but evaluate in a similar scene with 90 agents. Furthermore, some of the benchmarks like Narrow Corridor and Room Evacuation are quite different from the training dataset. DeepMNavigate is still able to compute collision-free and smooth trajectories for all the agents with no collisions and each agent arrives at its goal position. As shown in Fig. 2, the local learning method [16] does not consider the global map information and fails in such scenarios. In contrast, our approach enable robots to learn a reciprocal navigation behavior according to the global map information without any communication on action decision.

D. Quantitative Evaluation

We have evaluated the performance of our algorithm in terms of different evaluation metrics described above. In the circle crossing scenario, the failure rate of [16] rises with the increasing number or the density of robots. However, our approach always results in a stable performance and can

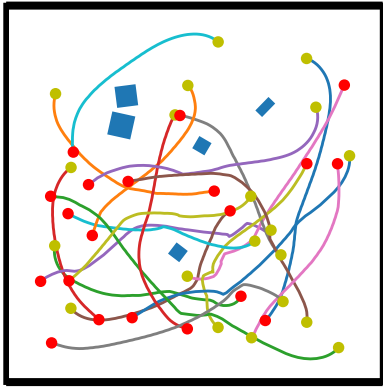


图 7: 随机起始位置和目标位置: 20 个机器人在有障碍物的场景中往返于随机位置的模拟轨迹。黄点是初始位置, 红点是最终位置。蓝色区域是位置和方向随机的障碍物。

r_{smooth} 值应该规范轨迹并使其更平滑。

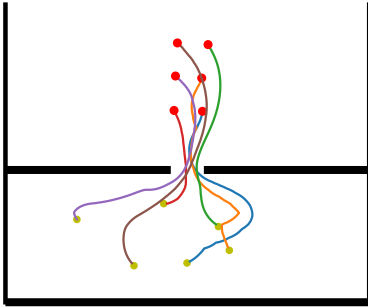


图 8: 房间疏散: 使用我们的算法模拟 6 个机器人疏散房间的轨迹。黄点是初始位置, 红点是最终位置。即使通道很窄, 我们的 DeepMNavigate 算法也能很好地工作。该基准与训练数据有很大不同。

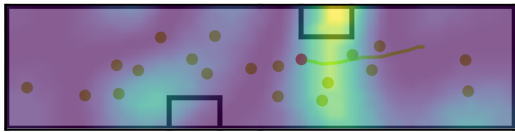


图 9: 在狭窄走廊基准中通过方法 [39] 生成的扰动显着性。红点代表当前代理, 黄点代表周围代理。黄色区域是行动政策的显着区域。

B. Evaluation Metrics and Benchmarks

为了评估导航算法的性能, 我们使用以下指标:

- *Success rate*: 在一定时间内达到目标且没有发生碰撞的机器人数量与环境中的机器人总数的比值。
- *Collision or stuck rate*: 如果机器人在限定时间内无法到达目的地或发生碰撞, 则分别被视为卡住或碰撞。

- *Extra time*: 所有机器人的平均行程时间与机器人行程时间下限之间的差值。后者计算为以最大速度直奔目标时的平均行进时间, 而不检查任何碰撞。

- *Average speed*: 所有机器人在导航过程中的平均速度。

我们在五个具有挑战性和代表性的基准中评估了我们的算法:

- *Circle Crossing*: 机器人均匀放置在一个圆周围, 每个机器人的目标位置根据直径位于圆的另一端。这些场景广泛应用于现有的多智能体导航算法[7]、[16] (图1)。
- *Narrow Corridor*: 两组机器人通过狭窄的走廊交换位置。这个基准对于几何分散方法 [7] 来说很困难, 因为它无法引导机器人穿过狭窄的通道 (图 2)。
- *Room with Obstacles*: 机器人穿过一个充满障碍物的房间, 从一侧到另一侧。仅使用局部信息的方法[16]将花费更多的时间来寻找到达目标的路径, 甚至可能由于缺乏全局信息而失败 (图6)。
- *Random Starts and Goals*: 机器人从随机初始位置出发, 移动到随机目标位置。此外, 还有位置和方向随机的障碍物。全局信息将有助于找到更安全、更快的路径 (图 7)。
- *Room Evacuation*: 机器人从随机初始位置出发, 撤离到房间外面。他们需要穿过一扇小门, 同时避免碰撞 (图8)。

C. New and Different Benchmarks

我们在基准上评估了我们的方法的性能, 这些基准在布局 and 包含狭窄段落方面与训练数据有很大不同。他们还使用不同数量的代理。在 Circle Crossing 基准测试中, 我们仅使用 12 个智能体进行训练, 但在类似场景中使用 90 个智能体进行评估。此外, 一些基准 (例如狭窄走廊和房间疏散) 与训练数据集有很大不同。DeepMNavigate 仍然能够计算所有智能体的无碰撞且平滑的轨迹, 没有碰撞, 并且每个智能体都到达其目标位置。如图2所示, 局部学习方法[16]没有考虑全局地图信息, 在这种情况下会失败。相比之下, 我们的方法使机器人能够根据全局地图信息学习交互导航行为, 而无需任何关于行动决策的通信。

D. Quantitative Evaluation

我们根据上述不同的评估指标评估了我们算法的性能。在环交叉场景中, [16]的故障率随着机器人数量或密度的增加而上升。然而, 我们的方法总是能带来稳定的性能, 并且可以

Metrics	Methods	# of agents (circle radius (unit:m))						
		30 (8)	40 (8)	50 (8)	60 (8)	70 (8)	80 (12)	90 (12)
Success Rate	[16]	1	0.975	0.96	0.95	0.929	0.7375	0.722
	Ours	1	1	1	1	0.986	1	1
Stuck/Collision Rate	[16]	0/0	0/0.025	0/0.04	0/0.05	0/0.071	0.175/0.0875	0.233/0.044
	Ours	0/0	0/0	0/0	0/0	0/0.014	0/0	0/0
Extra Time	[16]	4.32333	8.20256	7.8625	11.3088	13.54	15.1238	15.3292
	Ours	8.94667	8.73	9.738	10.1	12.4725	17.5525	34.1256
Average Speed	[16]	0.787272	0.661087	0.670508	0.585892	0.541638	0.54341	0.610233
	Ours	0.641368	0.646987	0.621649	0.613027	0.561946	0.506294	0.412899

Metrics	Methods	# of agents			
		8	12	16	20
Success Rate	[16]	0.875	0.75	0.5625	0.0
	Ours	1	1	1	1
Stuck/Collision Rate	[16]	0/0.125	0/0.25	0.3125/0.125	1/0
	Ours	0/0	0/0	0/0	0/0
Extra Time	[16]	4.8	12.5111	30.7222	-
	Ours	2.7	4.075	5.33125	8.36
Average Speed	[16]	0.653784	0.410699	0.230921	-
	Ours	0.742279	0.656969	0.594551	0.482519

Metrics	Methods	# of agents			
		5	10	15	20
Success Rate	[16]	1	0.7	0.6	0.7
	Ours	1	1	1	1
Stuck/Collision Rate	[16]	0/0	0.1/0.2	0.133/0.267	0.15/0.15
	Ours	0/0	0/0	0/0	0/0
Extra Time	[16]	9.55894	5.13058	6.38476	9.99585
	Ours	2.19487	2.55377	3.47301	7.80055
Average Speed	[16]	0.707441	0.734304	0.721682	0.508305
	Ours	0.858985	0.828276	0.775802	0.582426

TABLE II: The performance of our method (DeepMNavigate) and prior methods based on local information on different benchmarks (**Top: Circle Crossing; Bottom left: Narrow Corridor; Bottom right: Room with Obstacles.**), measured in terms of various metrics using different numbers of agents. The bold entries represent the best performance. Our method can guarantee a higher success rate in dense environments as compared to prior multi-agent navigation algorithms.

Metrics	Methods	# of agents			
		20	30	40	50
Success Rate	[16]	1	0.867	0.825	0.76
	Ours	1	1	1	1
Stuck/Collision Rate	[16]	0/0	0.033/0.1	0.05/0.125	0.2/0.04
	Ours	0/0	0/0	0/0	0/0
Extra Time	[16]	4.5974	9.38872	13.8348	12.6948
	Ours	3.59674	4.06355	10.4823	11.2948
Average Speed	[16]	0.601266	0.432238	0.354548	0.332425
	Ours	0.674891	0.63543	0.404211	0.383049

TABLE III: The performance of our proposed method and prior learning algorithms on the *Random Starts and Goals* benchmark. Our method demonstrates better results (bold face) than prior multi-agent navigation algorithms.

avoid the deadlock situation. At times, some of the robots may need to take a longer path to avoid congestion and this could reduce the robot's efficiency. As a result, we obtain better performance as compared to [16] or the decentralized collision avoidance methods in high density benchmarks like Circle Crossing.

Different from the circle crossing scenario, the other benchmarks incorporate some static obstacles in the environment. In this case, our method integrates the map information into the policy network and utilizes that information to handle such static obstacles and narrow passages. As the experimental results shown, our approach outperforms [16] both in terms of success rate and efficiency metric. In addition, our method behaves robustly even with the increasing density of robots.

Another important criteria to evaluate the performance of multi-robot systems is the stuck or collision rate, which is a measure of the number of robots cannot reach the goals or collide on the road. As shown in Tables II and III, our collision rate is zero for all these benchmarks. On the other hand, techniques based on local navigation information

result in some number of failures on different benchmarks. Furthermore, the failure increases as the number of agents or the density increases.

To better understand how the global information help the navigation system, we compute perturbation saliency over global map using method from [39] in the narrow corridor benchmark. We show the result in Fig. 9. The most important areas in the global map to help the decision making include: 1) agents in the front, which are blocked by other agents in local laser scan; 2) agents from the back, which could not be covered by local laser scan; 3) nearby obstacle. Thus, the global information can help the agent plan in advance, and be more alert to nearby obstacles.

E. Scalability

The running times for different numbers of robots are shown in Fig. 10. We can observe linear time performance with number of agents. That is because the decision process of our method is independent, each agent can compute their action by itself, based on the information it receives. Compared with most traditional geometric-based methods, a step analyzing the global environment may have super-linear time requirements, especially for the congested or challenging benchmarks used in this paper. For example, some methods compute K-nearest neighbors or the roadmap of the environment using the Voronoi diagram, which can have super-linear complexity. In contrast, our approach does not perform any such global computations and only leverages the power of the neural network. Moreover, the computation of the global map takes $O(n)$ time.

VI. CONCLUSION, LIMITATIONS, AND FUTURE WORK

We present a novel, multi-agent navigation algorithm based on deep reinforcement learning. We show that global

Metrics	Methods	# of agents (circle radius (unit:m))						
		30 (8)	40 (8)	50 (8)	60 (8)	70 (8)	80 (12)	90 (12)
Success Rate	[16]	1	0.975	0.96	0.95	0.929	0.7375	0.722
	Ours	1	1	1	1	0.986	1	1
Stuck/Collision Rate	[16]	0/0	0/0.025	0/0.04	0/0.05	0/0.071	0.175/0.0875	0.233/0.044
	Ours	0/0	0/0	0/0	0/0	0/0.014	0/0	0/0
Extra Time	[16]	4.32333	8.20256	7.8625	11.3088	13.54	15.1238	15.3292
	Ours	8.94667	8.73	9.738	10.1	12.4725	17.5525	34.1256
Average Speed	[16]	0.787272	0.661087	0.670508	0.585892	0.541638	0.54341	0.610233
	Ours	0.641368	0.646987	0.621649	0.613027	0.561946	0.506294	0.412899

Metrics	Methods	# of agents			
		8	12	16	20
Success Rate	[16]	0.875	0.75	0.5625	0.0
	Ours	1	1	1	1
Stuck/Collision Rate	[16]	0/0.125	0/0.25	0.3125/0.125	1/0
	Ours	0/0	0/0	0/0	0/0
Extra Time	[16]	4.8	12.5111	30.7222	-
	Ours	2.7	4.075	5.33125	8.36
Average Speed	[16]	0.653784	0.410699	0.230921	-
	Ours	0.742279	0.656969	0.594551	0.482519

Metrics	Methods	# of agents			
		5	10	15	20
Success Rate	[16]	1	0.7	0.6	0.7
	Ours	1	1	1	1
Stuck/Collision Rate	[16]	0/0	0.1/0.2	0.133/0.267	0.15/0.15
	Ours	0/0	0/0	0/0	0/0
Extra Time	[16]	9.55894	5.13058	6.38476	9.99585
	Ours	2.19487	2.55377	3.47301	7.80055
Average Speed	[16]	0.707441	0.734304	0.721682	0.508305
	Ours	0.858985	0.828276	0.775802	0.582426

表 II：我们的方法 (DeepMNavigate) 和基于不同基准上的本地信息的先前方法的性能（顶部： *Circle Crossing*；左下： *Narrow Corridor*；右下： *Room with Obstacles*。），使用不同数量的代理根据各种指标进行测量。粗体条目代表最佳性能。与现有的多智能体导航算法相比，我们的方法可以保证在密集环境中更高的成功率。

Metrics	Methods	# of agents			
		20	30	40	50
Success Rate	[16]	1	0.867	0.825	0.76
	Ours	1	1	1	1
Stuck/Collision Rate	[16]	0/0	0.033/0.1	0.05/0.125	0.2/0.04
	Ours	0/0	0/0	0/0	0/0
Extra Time	[16]	4.5974	9.38872	13.8348	12.6948
	Ours	3.59674	4.06355	10.4823	11.2948
Average Speed	[16]	0.601266	0.432238	0.354548	0.332425
	Ours	0.674891	0.63543	0.404211	0.383049

表 III：我们提出的方法和先前的学习算法在 *Random Starts and Goals* 基准上的性能。我们的方法展示了比之前的多智能体导航算法更好的结果（粗体）。

避免出现僵局的情况。有时，一些机器人可能需要走更长的路径以避免拥堵，这可能会降低机器人的效率。因此，与 [16] 或高密度基准（如 *Circle Crossing*）中的分散式碰撞避免方法相比，我们获得了更好的性能。

与圆圈交叉场景不同，其他基准测试包含环境中的一些静态障碍物。在这种情况下，我们的方法将地图信息集成到策略网络中，并利用该信息来处理此类静态障碍物和狭窄通道。正如实验结果所示，我们的方法在成功率和效率指标方面都优于[16]。此外，即使机器人密度不断增加，我们的方法也表现稳健。

评估多机器人系统性能的另一个重要标准是卡住或碰撞率，它是衡量无法到达目标或在道路上碰撞的机器人数量的指标。如表 II 和表 III 所示，所有这些基准的碰撞率均为零。另一方面，基于本地导航信息的技术

导致在不同的基准上出现一些失败。此外，随着代理数量或密度的增加，故障也会增加。

为了更好地理解全局信息如何帮助导航系统，我们在狭窄走廊基准中使用[39]中的方法计算全局地图上的扰动显着性。我们在图9中展示了结果。全局地图中帮助决策的最重要区域包括：1) 前面的智能体，在局部激光扫描中被其他智能体阻挡；2) 来自背面的药剂，局部激光扫描无法覆盖；3) 附近的障碍物。因此，全局信息可以帮助智能体提前规划，并对附近的障碍物更加警惕。

E. Scalability

不同数量的机器人的运行时间如图 10 所示。我们可以观察到代理数量的线性时间性能。这是因为我们方法的决策过程是独立的，每个代理都可以根据收到的信息自行计算其行动。与大多数传统的基于几何的方法相比，分析全局环境的步骤可能具有超线性时间要求，特别是对于本文中使用的拥挤或具有挑战性的基准。例如，一些方法使用 Voronoi 图计算 K 最近邻或环境路线图，这可能具有超线性复杂度。相比之下，我们的方法不执行任何此类全局计算，仅利用神经网络的力量。此外，全局地图的计算需要 $O(n)$ 时间。

六. 结论、局限性和未来的工作

我们提出了一种基于深度强化学习的新颖的多智能体导航算法。我们表明，全球

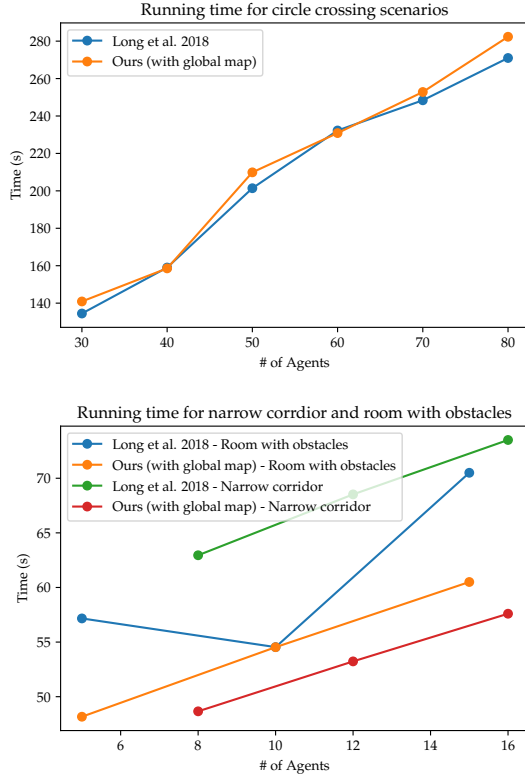


Fig. 10: We show the running time with different numbers of agents in several benchmarks. We use a CPU with 32 cores and NVIDIA RTX 2080 Ti to generate these performance graphs. This timing graph demonstrates that our approach is practical for many tens of robots. We also compare the running time with a learning-based algorithm that only uses local information [16]. The additional overhead in the running time with the use of the global information is rather small.

information about the environment can be used based on the global-map and present a novel network architecture to compute a collision-free trajectory for each robot. We highlight its benefits over many challenging scenarios and show benefits over geometric decentralized methods or reinforcement learning methods that only use local information. Moreover, our experimental results show that our DeepMNavigation algorithm can offer improved performance in dense and narrow scenarios, as compared to prior approaches. Furthermore, we demonstrate the performance on new, different benchmarks that are different from training scenarios. Overall, our approach demonstrates the value of global information in terms of discretized maps for DRL-based method.

Our results are promising and there are many ways to improve the performance. Our idea can be extended to 3D environments, by replacing the global map with a 3D version and using a 3D convolutional neural network to process it. Current training scenarios do not include dynamic and dense obstacles, so we may include them in the future work. Also, we need better techniques to compute the optimal size of the global-map and the local-map, and we can also include other components of the state information like velocity, orientation, or dynamics constraints. We also

need to extend the approach to handle general dynamic scenes where no information is available about the motion of the moving obstacles. Our approach assumes that the global information is available and that, in many scenarios, obtaining such information could be expensive. The use of global information increases the complexity of the training computation and we use a two-stage algorithm to reduce its running time. One other possibility is to use an auto-encoder to automatically derive the low-dimensional feature representations and then use them as a feature extractor. It may be possible to split the global and local navigation computation to combine our approach with local methods to avoid collisions with other agents or dynamic obstacles [7], [8]. Such a combination of local and global methods has been used to simulate large crowds [40] and it may be useful to develop a similar framework for learning-based algorithms. Furthermore, DRL-based methods that use local or global information can also be combined with global navigation data structures (e.g., roadmaps) to further improve navigation performance.

We have only demonstrated the application of our DRL-method on challenging, synthetic environments. A good area for future work is extending to real-world scenes, where we need to use other techniques to generate the motion information maps. It would be useful to combine our learning methods with SLAM techniques to improve the navigation capabilities.

VII. ACKNOWLEDGEMENT

This work was supported by ARO grants (W911NF1810313 and W911NF1910315) and Intel. Tingxiang Fan and Jia Pan were partially supported by HKSAR General Research Fund (GRF) HKU 11202119, 11207818.

REFERENCES

- [1] J. P. Van Den Berg and M. H. Overmars, "Prioritized motion planning for multiple robots," in *ROS*. IEEE, 2005, pp. 430–435.
- [2] J. van Den Berg, J. Snoeyink, M. C. Lin, and D. Manocha, "Centralized path planning for multiple robots: Optimal decoupling into sequential plans," in *Robotics: Science and systems*, vol. 2, 2009.
- [3] R. J. Luna and K. E. Bekris, "Push and swap: Fast cooperative path-finding with completeness guarantees," in *IJCAI*, 2011.
- [4] G. Sanchez and J.-C. Latombe, "Using a prm planner to compare centralized and decoupled planning for multi-robot systems," in *ICRA*, vol. 2. IEEE, 2002, pp. 2112–2119.
- [5] K. Solovey and D. Halperin, "On the hardness of unlabeled multi-robot motion planning," *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1750–1759, 2016.
- [6] J. Yu and D. Rus, "An effective algorithmic framework for near optimal multi-robot path planning," in *Robotics Research*. Springer, 2018, pp. 495–511.
- [7] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics research*. Springer, 2011.
- [8] R. Geraets, A. Kamphuis, I. Karamouzas, and M. Overmars, "Using the corridor map method for path planning for a large number of characters," in *International Workshop on Motion in Games*, 2008.
- [9] D. Helbing and P. Molnar, "Social force model for pedestrian dynamics," *Physical review E*, vol. 51, no. 5, p. 4282, 1995.
- [10] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, 1997.
- [11] T. Fraichard and H. Asama, "Inevitable collision states: a step towards safer robots?" *Advanced Robotics*, 2004.

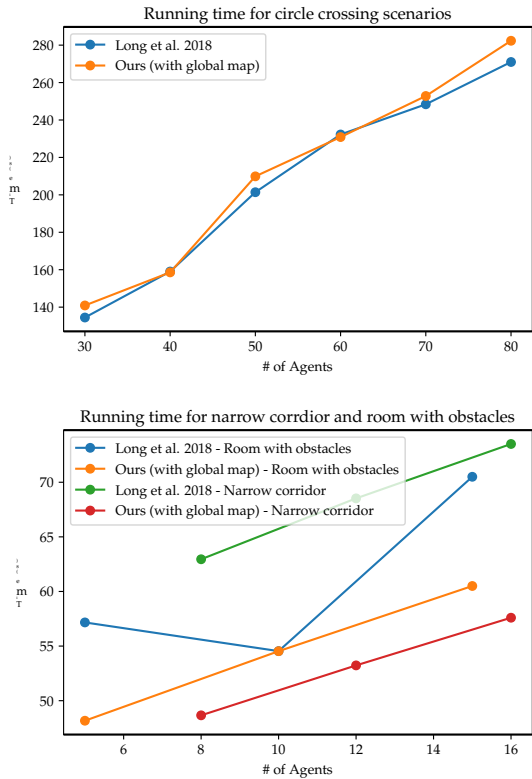


图 10: 我们展示了多个基准测试中不同数量的代理的运行时间。我们使用 32 核 CPU 和 NVIDIA RTX 2080 Ti 来生成这些性能图。该时序图表明我们的方法对于数十个机器人来说是实用的。我们还将运行时间与仅使用本地信息的基于学习的算法进行了比较[16]。使用全局信息的运行时间的额外开销相当小。

可以基于全局地图使用有关环境的信息，并提出一种新颖的网络架构来计算每个机器人的无碰撞轨迹。我们强调了它在许多具有挑战性的场景中的好处，并展示了它相对于几何分散方法或仅使用本地信息的强化学习方法的好处。此外，我们的实验结果表明，与之前的方法相比，我们的 DeepMNavigation 算法可以在密集和狭窄的场景中提供更好的性能。此外，我们还展示了与训练场景不同的新的不同基准的性能。总体而言，我们的方法证明了基于 DRL 方法的离散映射的全局信息的价值。

我们的结果很有希望，并且有很多方法可以提高性能。我们的想法可以扩展到 3D 环境，通过用 3D 版本替换全局地图并使用 3D 卷积神经网络来处理它。目前的训练场景不包括动态和密集的障碍物，因此我们可能会将它们包含在未来的工作中。此外，我们需要更好的技术来计算全局地图和局部地图的最佳大小，并且我们还可以包含状态信息的其他组成部分，例如速度、方向或动态约束。我们也

需要扩展该方法来处理一般动态场景，其中没有有关移动障碍物运动的信息。我们的方法假设全局信息可用，并且在许多情况下，获取此类信息可能会很昂贵。全局信息的使用增加了训练计算的复杂性，我们使用两阶段算法来减少其运行时间。另一种可能性是使用自动编码器自动导出低维特征表示，然后将它们用作特征提取器。可以将全局和局部导航计算分开，将我们的方法与局部方法结合起来，以避免与其他代理或动态障碍物发生碰撞[7]、[8]。这种局部和全局方法的组合已被用来模拟大型人群[40]，并且为基于学习的算法开发类似的框架可能很有用。此外，使用局部或全局信息的基于 DRL 的方法也可以与全局导航数据结构（例如路线图）相结合，以进一步提高导航性能。

我们仅演示了 DRL 方法在具有挑战性的合成环境中的应用。未来工作的一个好领域是扩展到现实世界场景，我们需要使用其他技术来生成运动信息图。将我们的学习方法与 SLAM 技术相结合以提高导航能力将很有用。

七. 致谢

这项工作得到了 ARO 拨款（W911NF1810313 和 W911NF1910315）和英特尔的支持。范廷祥和潘佳获得香港特别行政区普通研究基金（GRF）HKU 11202119、11207818 的部分支持。

参考

- [1] J. P. Van Den Berg 和 M. H. Overmars, “多个机器人的优先运动规划”, *IROS*. IEEE, 2005 年, 第 430–435 页。
- [2] J. van Den Berg, J. Snoeyink, M. C. Lin 和 D. Manocha, “多个机器人的集中路径规划: 优化解耦为顺序计划。” 在 *Robotics: Science and systems*, 卷。 [3] R. J. Luna 和 K. E. Bekris, “推送和交换: 具有完整性保证的快速协作寻路”, *IJCAI*, 2011 年。
- [4] G. Sanchez 和 J.-C. 2, 2009 年。 Latombe, “使用 prm 规划器比较多机器人系统的集中式和解耦规划”, *ICRA*, 卷。 2. IEEE, 2002 年, 第 2112–2119 页。
- [5] K. Solovey 和 D. Halperin, “论无标签多机器人运动规划的难度”, *The International Journal of Robotics Research*, 卷。 35, 没有。 14, 第 1750–1759 页, 2016 年。
- [6] J. Yu 和 D. Rus, “一种用于近乎最优多机器人路径规划的有效算法框架”, *Robotics Research*. 施普林格, 2018 年, 第 495–511 页。
- [7] J. Van Den Berg, S. J. Guy, M. Lin 和 D. Manocha, “相互 n 体碰撞避免”, 参见 *Robotics research*. Springer, 2011。
- [8] R. Geraerts, A. Kamphuis, I. Karamouzas 和 M. Overmars, “使用走廊地图方法对大量角色进行路径规划”, *International Workshop on Motion in Games*, 2008。
- [9] D. Helbing 和 P. Molnar, “行人动力学的社会力模型”, *Physical review E*, 卷。 51, 没有。 5, p. 4282, 1995。
- [10] D. Fox, W. Burgard 和 S. Thrun, “避免碰撞的动态窗口方法”, *IEEE Robotics & Automation Magazine*, 1997。
- [11] T. Fraichard 和 H. Asama, “不可避免的碰撞是迈向更安全机器人的一步?” *Advanced Robotics*, 2004 年。

- [12] T. Fan, P. Long, W. Liu, and J. Pan, "Fully distributed multi-robot collision avoidance via deep reinforcement learning for safe and efficient navigation in complex scenarios," *arXiv*, 2018.
- [13] M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart, and C. Cadena, "From Perception to Decision: A Data-driven Approach to End-to-end Motion Planning for Autonomous Ground Robots," *arXiv e-prints*, p. arXiv:1609.07910, Sep 2016.
- [14] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *ICRA*. IEEE, 2017, pp. 285–292.
- [15] M. Everett, Y. F. Chen, and J. P. How, "Motion planning among dynamic, decision-making agents with deep reinforcement learning," in *IROS*. IEEE, 2018, pp. 3052–3059.
- [16] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *ICRA*. IEEE, 2018, pp. 6252–6259.
- [17] P. Long, W. Liu, and J. Pan, "Deep-learned collision avoidance policy for distributed multiagent navigation," *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 656–663, 2017.
- [18] R. Luna and K. E. Bekris, "Efficient and complete centralized multi-robot path planning," in *IROS*. IEEE, 2011, pp. 3268–3275.
- [19] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [20] J. Yu and S. M. LaValle, "Optimal multirobot path planning on graphs: Complete algorithms and effective heuristics," *IEEE Transactions on Robotics*, vol. 32, no. 5, pp. 1163–1177, 2016.
- [21] S. Tang, J. Thomas, and V. Kumar, "Hold or take optimal plan (hoop): A quadratic programming approach to multi-robot trajectory generation," *IJRR*, vol. 37, no. 9, pp. 1062–1084, 2018.
- [22] J. Alonso-Mora, A. Breitenmoser, M. Ruffi, P. Beardsley, and R. Siegwart, "Optimal reciprocal collision avoidance for multiple non-holonomic robots," in *Distributed Autonomous Robotic Systems*. Springer, 2013, pp. 203–216.
- [23] U. Muller, J. Ben, E. Cosatto, B. Flepp, and Y. L. Cun, "Off-road obstacle avoidance through end-to-end learning," in *Advances in neural information processing systems*, 2006, pp. 739–746.
- [24] J. Zhang, J. T. Springenberg, J. Boedecker, and W. Burgard, "Deep reinforcement learning with successor features for navigation across similar environments," in *IROS*. IEEE, 2017, pp. 2371–2378.
- [25] A. Barreto, W. Dabney, R. Munos, J. J. Hunt, T. Schaul, H. P. van Hasselt, and D. Silver, "Successor features for transfer in reinforcement learning," in *NIPS*, 2017, pp. 4055–4065.
- [26] J. Sergeant, N. Sünderhauf, M. Milford, and B. Upcroft, "Multimodal deep autoencoders for control of a mobile robot," in *Proc. of Australasian Conf. for Robotics and Automation (ACRA)*, 2015.
- [27] S. Ross, N. Melik-Barkhudarov, K. S. Shankar, A. Wendel, D. Dey, J. A. Bagnell, and M. Hebert, "Learning monocular reactive uav control in cluttered natural environments," in *ICRA*. IEEE, 2013.
- [28] M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart, and C. Cadena, "From perception to decision: A data-driven approach to end-to-end motion planning for autonomous ground robots," in *ICRA*, 2017.
- [29] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *IROS*. IEEE, 2017, pp. 31–36.
- [30] G. Kahn, A. Villafior, V. Pong, P. Abbeel, and S. Levine, "Uncertainty-aware reinforcement learning for collision avoidance," *arXiv*, 2017.
- [31] J. Godoy, I. Karamouzas, S. J. Guy, and M. L. Gini, "Moving in a crowd: Safe and efficient navigation among heterogeneous agents," in *IJCAI*, 2016, pp. 294–300.
- [32] Y. F. Chen, M. Everett, M. Liu, and J. P. How, "Socially aware motion planning with deep reinforcement learning," in *IROS*, 2017.
- [33] H.-J. Yoon, H. Chen, K. Long, H. Zhang, A. Gahlawat, D. Lee, and N. Hovakimyan, "Learning to communicate: A machine learning framework for heterogeneous multi-agent robotic systems," in *AIAA Scitech 2019 Forum*, 2019, p. 1456.
- [34] A. Jagan Sathyamoorthy, J. Liang, U. Patel, T. Guan, R. Chandra, and D. Manocha, "Densecavoid: Real-time navigation in dense crowds using anticipatory behaviors," *arXiv*, pp. arXiv–2002, 2020.
- [35] Wikipedia contributors, "Algorithms for calculating variance — Wikipedia, the free encyclopedia," 2020, [Online; accessed 31-May-2020]. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Algorithms_for_calculating_variance&oldid=959752885
- [36] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry, "Implementation matters in deep rl: A case study on ppo and trpo," in *ICLR*, 2020.
- [37] A. Nair, B. McGrew, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Overcoming exploration in reinforcement learning with demonstrations," in *ICRA*. IEEE, 2018, pp. 6292–6299.
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv*, 2017.
- [39] S. Greydanus, A. Koul, J. Dodge, and A. Fern, "Visualizing and understanding atari agents," *arXiv preprint arXiv:1711.00138*, 2017.
- [40] R. Narain, A. Golas, S. Curtis, and M. C. Lin, "Aggregate dynamics for dense crowd simulation," in *TOG*. ACM, 2009.

[12] T. Fan, P. Long, W. Liu 和 J. Pan, “通过深度强化学习实现完全分布式多机器人碰撞避免, 在复杂场景中实现安全高效的导航”, *arXiv*, 2018. [13] M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart 和 C. Cadena, “从感知到决策: 一种数据驱动的端到端运动方法”自主地面机器人规划, ” *arXiv e-prints*, 第 14 页. arXiv:1609.07910, 2016 年 9 月. [14] Y. F. Chen, M. Liu, M. Everett 和 J. P. How, “通过深度强化学习实现去中心化非通信多智能体碰撞避免”, 见 *ICRA*. IEEE, 2017 年, 第 285–292 页. [15] M. Everett, Y. F. Chen 和 J. P. How, “采用深度强化学习的动态决策代理之间的运动规划”, 参见 *IROS*. IEEE, 2018 年, 第 3052–3059 页. [16] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang 和 J. Pan, “通过深度强化学习实现最佳去中心化多机器人碰撞避免”, *ICRA*. IEEE, 2018 年, 第 6252–6259 页. [17] P. Long, W. Liu, J. Pan, “分布式多智能体导航的深度学习碰撞避免策略”, *IEEE Robotics and Automation Letters*, 卷。2、没有。2, 第 656–663 页, 2017 年. [18] R. Luna 和 K. E. Bekris, “高效且完整的集中式多机器人路径规划”, *IROS*. IEEE, 2011 年, 第 3268–3275 页. [19] G. Sharon, R. Stern, A. Felner 和 N. R. Sturtevant, “基于冲突的最佳多智能体寻路搜索”, *Artificial Intelligence*, 卷。219, 第 40–66 页, 2015 年. [20] J. Yu 和 S. M. LaValle, “图上的最优多机器人路径规划: 完整算法和有效启发式”, *IEEE Transactions on Robotics*, 卷。32、没有。5, 第 1163–1177 页, 2016 年. [21] S. Tang, J. Thomas 和 V. Kumar, “保持或采取最佳计划 (环): 多机器人轨迹生成的二次规划方法”, *IJRR*, 卷。37、没有。9, 第 1062–1084 页, 2018 年. [22] J. Alonso-Mora, A. Breitenmoser, M. Rufli, P. Beardsley 和 R. Siegwart, “多个非完整机器人的最佳相互碰撞避免”, *Distributed Autonomous Robotic Systems*. 施普林格, 2013 年, 第 203–216 页. [23] U. Muller, J. Ben, E. Cosatto, B. Flepp 和 Y. L. Cun, “通过端到端学习实现越野避障”, *Advances in neural information processing systems*, 2006 年, 第 739–746 页. [24] J. 张, J. T. Springenberg, J. Boedecker 和 W. Burgard, “具有后继特征的深度强化学习, 用于在类似环境中导航”, *IROS*. IEEE, 2017 年, 第 2371–2378 页. [25] A. Barreto, W. Dabney, R. Munos, J. J. Hunt, T. Schaul, H. P. van Hasselt 和 D. Silver, “强化学习中迁移的后继特征”, *NIPS*, 2017 年, 第 4055–4065 页. [26] J. Sergeant, N. Sünderhauf, M. Milford 和 B. Upcroft, “用于控制移动机器人的多模式深度自动编码器”, 载于 *Proc. of Australasian Conf. for Robotics and Automation (ACRA)*, 2015 年. [27] S. Ross, N. Melik-Barkhudarov, K. S. Shankar, A. Wendel, D. Dey, J. A. Bagnell 和 M. Hebert, “在杂乱的自然环境中学习单目反应式无人机控制”, *ICRA*. IEEE, 2013 年. [28] M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart 和 C. Cadena, “从感知到决策: 自主地面机器人端到端运动规划的数据驱动方法”, *ICRA*, 2017 年. [29] L. Tai, G. Paolo 和 M. Liu, “虚拟到真实的深度强化学习: 持续控制用于无地图导航的移动机器人”, *IROS*. IEEE, 2017 年, 第 31–36 页. [30] G. Kahn, A. Villafior, V. Pong, P. Abbeel 和 S. Levine, “用于避免碰撞的不确定性感知强化学习”, *arXiv*, 2017. [31] J. Godoy, I. Karamouzas, S. J. Guy 和 M. L. Gini, “在人群中移动: 异构智能体之间安全高效的导航。”见 *IJCAI*, 2016 年, 第 294–300 页. [32] Y. F. Chen, M. Everett, M. Liu 和 J. P. How, “利用深度强化学习进行社会意识运动规划”, *IROS*, 2017 年. [33] H.-J. Yoon, H. Chen, K. Long, H. Zhang, A. Gahlawat, D. Lee 和 N. Hovakimyan, “学习通信: 异构多智能体机器人系统的机器学习框架”, *AIAA Scitech 2019 Forum*, 2019 年, 第 17 页. 1456. [34] A. Jagan Sathyamoorthy, J. Liang, U. Patel, T. Guan, R. Chandra 和 D. Manocha, “Densecavoid: 使用预期行为在密集人群中进行实时导航”, *arXiv*, 第 arXiv–2002, 2020. [35] 维基百科贡献者, “计算方差的算法 — 维基百科, 免费百科全书,” 2020, [在线; 2020 年 5 月 31 日访问]. [在线的]. 可用: <https://en.wikipedia.org/w/index.php?title=计算方差&oldid=959752885>的算法 [36] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph,

和 A. Madry, “深度 rl 中的实现很重要: ppo 和 trpo 的案例研究”, *ICLR*, 2020. [37] A. Nair, B. McGrew, M. Andrychowicz, W. Zaremba 和 P. Abbeel, “通过演示克服强化学习中的探索”, *ICRA*. IEEE, 2018 年, 第 6292–6299 页. [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford 和 O. Klimov, “近端策略优化算法”, *arXiv*, 2017 年. [39] S. Greydanus, A. Koul, J. Dodge 和 A. Fern, “可视化和理解 atari 代理”, *arXiv preprint arXiv:1711.00138*, 2017 年. [40] R. Narain, A. Golas, S. Curtis 和 M. C. Lin, “密集人群模拟的聚合动力学”, *TOG*. 美国CM, 2009.